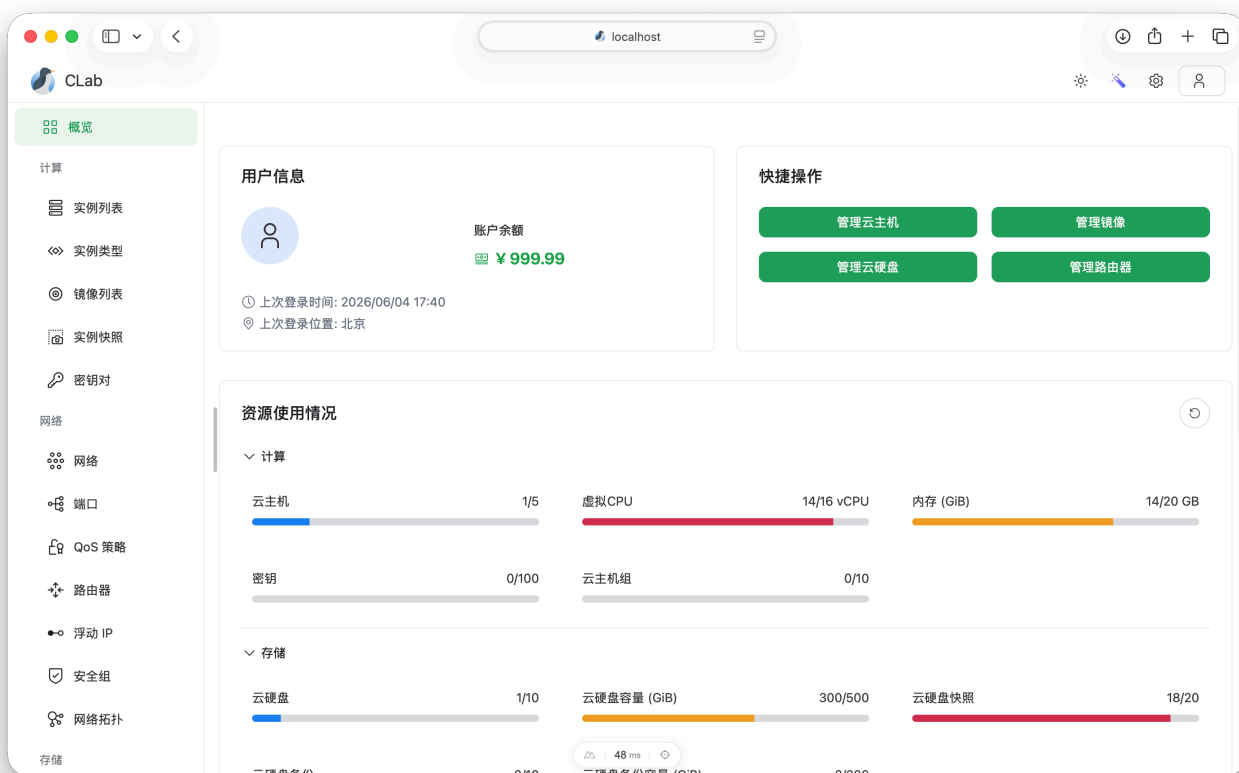


CLab v2 新特性简介

CLab v2 就是为了解决 Clab 的一些问题而立项的。Clab v2 把计算、存储、网络、GPU 和运维能力重新组织成更接近真实云平台的科研基础设施。它面向的不是单次演示，而是更长期、更稳定、更可复现的教学与科研使用。很多能力目前仍在 PoC 或调试阶段，但方向已经很清楚：让物理服务器上的 CPU、内存、GPU、网卡和存储，变成可调度、可隔离、可运维的云资源。

前端更新

现代化的前端，接入AI助手。



计算：从云主机到统一资源管理

v2 的 Compute 方向负责虚拟机、CPU/GPU 虚拟化、裸机虚拟化和高性能网络能力。短期仍保留 OpenStack 作为兼容路线，因为它已有 Nova、libvirt、Glance、Cinder 等部署和调试经验；但长期主线会转向 Kubernetes / KubeVirt。

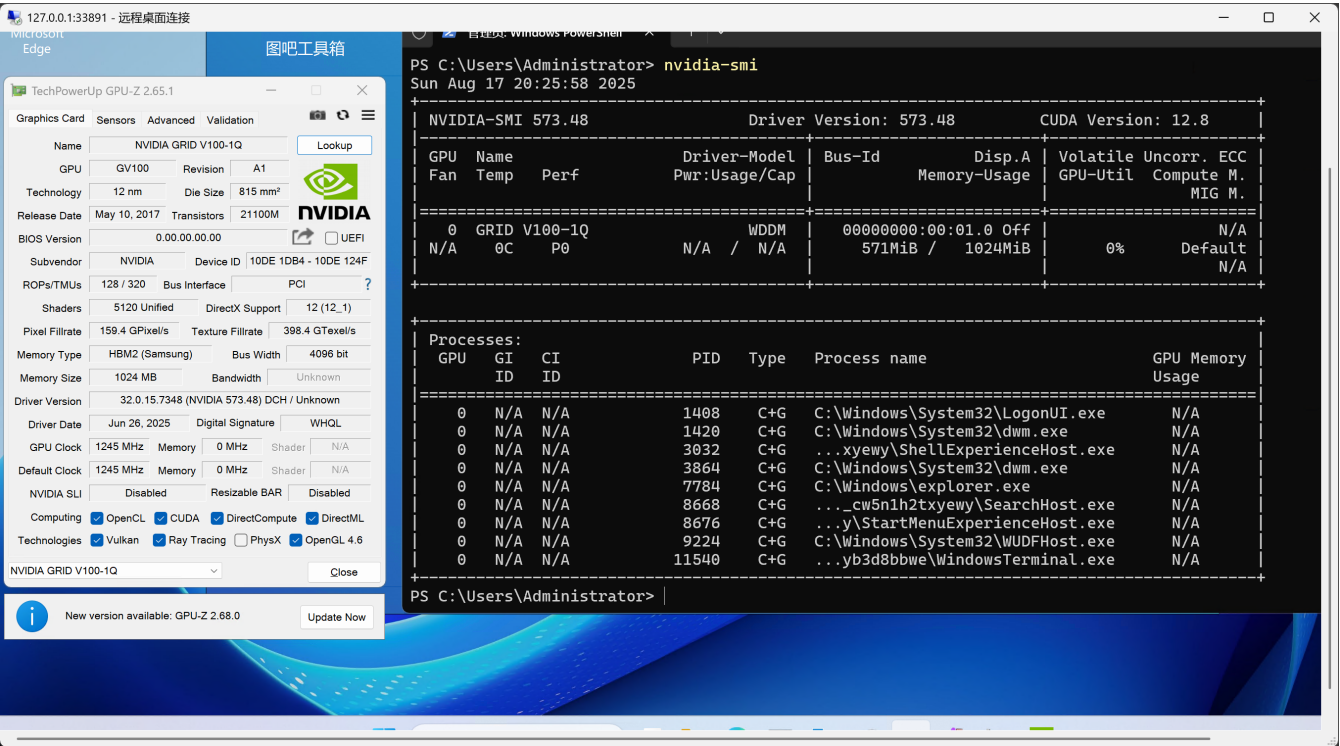
KubeVirt 可以理解为“让虚拟机也成为 Kubernetes 管理的资源”。这样，云主机不再只是某台远程机器，而是可以被集群统一调度、监控和恢复的计算对象。v2 计划先跑通最小 VM 链路：创建虚拟机、进入 console、挂载存储、接入网络，再逐步接入 GPU、RDMA 和裸机能力。

v2 也开始关注 CPU 与 VM 性能。虚拟机能跑只是第一步，更重要的是知道它相比宿主机慢多少、慢在哪里。后续会建立宿主机、裸 QEMU、OpenStack VM、KubeVirt VM 的性能基线，分析 CPU、内存、NUMA、磁盘和网络路径上的开销。

GPU：从独占显卡到可管理资源

GPU 是 v2 面向科研最重要的新能力之一。深度学习、图像分析、蛋白结构预测、分子模拟、材料计算都可能需要 GPU。v2 的 GPU 虚拟化路线分三类。

第一类是 PCIe 直通 / VFIO，把一张物理 GPU 直接交给一台虚拟机使用，适合短期 PoC 和高性能独占任务。第二类是 vGPU / SR-IOV，把 GPU 切分给多个虚拟机，适合共享和图形桌面，但受硬件、驱动和授权限制。第三类是 Remote GPU Pool / virtio-vcuda，目标是让纯 CPU 虚拟机也能远程使用 GPU，并支持调度和一定程度的池化。



这些能力的核心意义是：GPU 不再只是“某台服务器上的特殊硬件”，而是逐步变成平台可以申请、分配、回收和计量的资源。

存储：数据进入共享存储体系

v2 的 Storage 方向强调高性能共享存储和长期运维。当前 store / SigStore 已完成硬件整備、Ceph 调研、正式部署，上线了 RGW 和 RBD，并进入稳定运行与调优阶段。

对科研同学来说，块存储可以理解为挂到云主机上的数据盘，对象存储更像云端文件仓库。这样，实验数据、模型权重、模拟轨迹和中间结果不必跟着某台云主机一起消失，也更适合多人协作。

存储方向还在处理 RDMA 回归、PG balance、故障演练、quota、多租户访问控制、Cinder 迁移等问题。它们看起来偏运维，但本质上是在回答一个科研平台必须面对的问题：硬盘坏了、网络抖了、数据变多了、多人同时访问时，数据还能不能可靠保存。

未来的 DAOS 更偏高性能并行存储，适合 NVMe、高并发读写和大规模计算场景。它不属于 Compute，而是 Storage 的长期探索方向。

RDMA 与高性能网络：让多机协作更快

当任务只在一台机器上跑时，普通网络通常够用；但多节点模拟、分布式训练、大规模数据处理会很快遇到网络瓶颈。v2 的 RDMA / 高性能网络方向，就是让虚拟机和容器也尽量获得接近裸机的低延迟、高带宽网络。

当前主线是 SR-IOV + Multus + KubeVirt：通过 device plugin 发现和分配网卡 VF，通过 Kubernetes resource、CDI 和 NetworkAttachmentDefinition 把设备交给容器或虚拟机。容器侧通常使用 `mlx5_core`，虚拟机侧使用 `vfio-pci`。后续还要继续完善 VF 资源池、driver 切换、健康检查、PFC / ECN / QoS 等能力。

对物理、化学、生物等科研任务来说，这类能力的价值在于：未来多台云主机协作时，数据交换不再轻易成为最慢的一环。

裸机虚拟化与 DPU：把整台服务器纳入云平台

有些任务对虚拟化开销非常敏感，或者需要特殊硬件和启动方式，普通虚拟机不一定合适。v2 的裸机虚拟化方向希望把完整物理服务器也作为云资源交付。

这里的关键是 DPU。v2 的设想是：DPU 侧虚拟化网络，让 Host 看到已经接入 VPC 的网口；DPU 侧访问远端存储，并把它暴露成块设备或 NVMe；裸机从 DPU 提供的存储启动；平台再通过 IPMI 控制服务器开机、关机和重启。同时，Host 侧需要封锁 IPMI，并让 DPU 进入 zero trust 模式。

这部分目前更接近设计和 PoC，但它说明 v2 不只是管理虚拟机，也希望把从云主机到整台物理服务器的不同计算形态都纳入统一体系。

对科研同学的意义

[一个使用 CLab 构建的实例](#)

CLab v2 的新特性可以对应很多科研需求：物理中的蒙特卡洛模拟、有限元和流体计算；化学中的分子动力学、量子化学、分子 docking 和材料模拟；生物中的测序分析、蛋白结构预测、显微图像处理和模型训练；交叉学科中的大规模矩阵计算、优化、随机模拟和 AI 辅助科研。

这些场景共同需要更强的计算能力、更可靠的数据管理、更清晰的资源分配和更好的复现条件。CLab v2 提供的不是最终答案，而是让想法变大的基础设施：让一个小程序有机会变成可复现的实验，让一次手动运行变成一组系统的参数扫描，让个人电脑上的结果进入团队可以共同使用的数据和环境。

这也和数据结构与算法课有关。算法决定任务怎么计算，数据结构决定状态怎么保存，云平台决定这些计算能否被放大、复现和协作。对非计算机专业同学来说，理解 CLab v2 不需要成为云计算工程师；只需要知道，它正在把计算资源变成更像实验仪器一样可申请、可使用、可维护的科研工具。

如何更好的使用 CLab

了解 Linux 的基础知识

想要更为详尽的了解 Linux 知识可以参考 LCPU 编写的 [LCPU Getting Started 计算机入门系列课程](#)

包管理器

对于使用 Windows 的同学，习惯的软件下载方式可能是在网络上搜索并下载打包好的安装程序（通常为 `.exe` 或 `.msi`），对于使用 macOS 的同学，可能是在 App Store 下载或者是在网络上搜索并下载 `.dmg` 镜像挂载后拖入 Application 文件夹。

但是对于目前只能以 `ssh` 访问的 CLab 来说，我们想要安装软件只能通过终端进行，我们以 `Codex cli` 为例：

<https://developers.openai.com/codex/quickstart?setup=cli>

要么通过 curl 抓取并执行下载脚本，要么从包管理器下载。

常见系统包管理器和语言包管理器汇总

—— 快速了解平台、用途与常用命令 ——



系统包管理器： 管理操作系统级软件、依赖库、命令行工具。



语言包管理器： 管理特定编程语言的库、框架与项目依赖。



一、系统包管理器

名称	常见平台	常用命令示例	说明
apt	Debian / Ubuntu	<code>sudo apt install nginx</code>	最常见的 Debian 系发行版包管理器
dnf	Fedora / 新版 RHEL	<code>sudo dnf install git</code>	yum 的后继者，依赖处理更现代
yum	CentOS / 旧版 RHEL	<code>sudo yum install wget</code>	经典的 RPM 系包管理工具
pacman	Arch Linux	<code>sudo pacman -S python</code>	简洁高效，Arch 生态核心工具
zypper	openSUSE	<code>sudo zypper install vim</code>	适用于 SUSE / openSUSE
Homebrew	macOS / Linux	<code>brew install node</code>	开发者常用，macOS 上尤其流行
winget	Windows	<code>winget install Python.Python.3</code>	微软官方命令行包管理器
Chocolatey	Windows	<code>choco install vscode</code>	Windows 常见第三方包管理器
Scoop	Windows	<code>scoop install gcc</code>	轻量、适合命令行用户



二、语言包管理器

名称	语言 / 生态	常用命令示例	说明
pip	Python	<code>pip install requests</code>	Python 最常用的包管理器
conda	Python / 数据科学	<code>conda install numpy</code>	兼顾环境管理与包管理
npm	JavaScript / Node.js	<code>npm install express</code>	前端与 Node.js 生态核心工具
yarn	JavaScript	<code>yarn add react</code>	常用于前端工程化
pnpm	JavaScript	<code>pnpm add vue</code>	节省磁盘空间，安装速度快
cargo	Rust	<code>cargo add serde</code>	Rust 官方构建与包管理工具
gem	Ruby	<code>gem install rails</code>	RubyGems 包管理器
composer	PHP	<code>composer require laravel/framework</code>	PHP 依赖管理标准工具
Maven	Java	<code>mvn dependency:resolve</code>	Java 经典项目与依赖管理工具
Gradle	Java / Kotlin	<code>gradle build</code>	现代化构建工具，也管理依赖
NuGet	.NET / C#	<code>dotnet add package Newtonsoft.Json</code>	.NET 生态标准包管理器
Go Modules	Go	<code>go get github.com/gin-gonic/gin</code>	Go 官方依赖管理机制



如何区分？

- ✓ 系统包管理器：面向操作系统和全局软件安装
- ✓ 语言包管理器：面向项目依赖与开发生态
- ✓ 很多开发环境会同时使用两类工具



提示： 实际使用时请结合发行版、项目规范和虚拟环境选择工具。

用 Conda (Anaconda) 管理 Python 版本和环境

在 Clab 上跑科研代码时，最常见的问题不是代码本身，而是 Python 版本和库版本对不上。Conda 的作用就是把不同项目的 Python 环境隔离开：这个项目用 Python 3.10 和 PyTorch，另一个项目用 Python 3.11 和 NumPy，它们互不影响。

在系统中安装 conda：

```
1 | sudo dnf install conda # fedora
```

常用指令如下。

查看当前有哪些环境：

```
1 | conda env list
```

创建一个新环境，并指定 Python 版本：

```
1 | conda create -n myenv python=3.11
```

进入环境：

```
1 | conda activate myenv
```

退出当前环境：

```
1 | conda deactivate
```

在当前环境中安装包：

```
1 | conda install numpy pandas matplotlib
```

如果 conda 里没有需要的包，可以在进入环境后使用 pip：

```
1 | pip install package-name
```

建议优先用 `conda install`，找不到包时再用 `pip install`。不要在没有激活环境时随便装包，否则容易把系统 Python 或 base 环境弄乱。

查看当前环境中已经安装的包：

```
1 | conda list
```

更新某个包：

```
1 | conda update numpy
```

删除某个包：

```
1 | conda remove numpy
```

删除整个环境：

```
1 | conda remove -n myenv --all
```

把当前环境导出为配置文件，方便之后复现：

```
1 | conda env export > environment.yml
```

用 `environment.yml` 恢复环境：

```
1 | conda env create -f environment.yml
```

如果只想快速记录核心依赖，也可以手写一个较简洁的 `environment.yml`：

```
1 | name: myenv
2 | channels:
3 |   - conda-forge
4 | dependencies:
5 |   - python=3.11
6 |   - numpy
7 |   - pandas
8 |   - matplotlib
9 |   - pip
```

实际使用时建议每个项目单独建一个环境，例如：

```
1 | conda create -n bio-project python=3.10
2 | conda activate bio-project
3 | conda install numpy scipy pandas matplotlib
```

这样即使某个项目的库版本装坏了，也只需要删除这个环境重新创建，不会影响其他项目。

用 uv 管理 Python 版本和环境

在系统中安装 uv：

```
1 | sudo dnf install uv # fedora等红帽系Distro
```

`uv` 是一个更现代、更快的 Python 项目和环境管理工具。它可以管理 Python 版本、虚拟环境、依赖、锁文件，也可以替代一部分 `pip`、`venv`、`pipx` 的工作。和 Conda 相比，`uv` 更适合纯 Python 项目；如果项目依赖很多非 Python 的系统库，Conda 仍然更稳。

查看 uv 版本：

```
1 | uv --version
```

安装指定 Python 版本：

```
1 | uv python install 3.11
```

查看 uv 能找到哪些 Python：

```
1 | uv python list
```

在当前目录初始化一个 Python 项目：

```
1 | uv init
```

指定 Python 版本初始化项目：

```
1 | uv init --python 3.11
```

初始化后，项目中通常会出现 `pyproject.toml`。这是记录项目依赖和配置的文件，类似 Conda 项目里的 `environment.yml`，但更偏 Python 标准项目格式。

给项目添加依赖：

```
1 | uv add numpy pandas matplotlib
```

添加开发依赖，例如格式化、测试工具：

```
1 | uv add --dev pytest ruff
```

删除依赖：

```
1 | uv remove numpy
```

根据 `pyproject.toml` 和 `uv.lock` 同步环境：

```
1 | uv sync
```

运行项目中的 Python 文件：

```
1 | uv run python main.py
```

运行一条临时 Python 命令：

```
1 | uv run python -c "import sys; print(sys.version)"
```

如果只是想像 `pip install` 一样在当前环境里装包，可以使用：

```
1 | uv pip install numpy
```

不过在 uv 项目里更推荐使用 `uv add`，因为它会把依赖写进 `pyproject.toml` 并更新锁文件，方便之后复现环境。

把已有 `requirements.txt` 安装到当前环境：

```
1 | uv pip install -r requirements.txt
```

把依赖锁定到 `uv.lock`：

```
1 | uv lock
```

运行命令行工具但不全局安装，例如临时运行 `ruff`：

```
1 | uvx ruff check .
```

如果你从 GitHub 或别人那里拿到一个 uv 项目，通常只需要：

```
1 uv sync
2 uv run python main.py
```

一个简单的科研脚本项目可以这样开始：

```
1 mkdir simulation-demo
2 cd simulation-demo
3 uv init --python 3.11
4 uv add numpy matplotlib
5 uv run python main.py
```

简单来说：如果只是做课程作业或科研脚本，`uv init`、`uv add`、`uv sync`、`uv run` 这四个命令基本够用；如果需要 CUDA、复杂编译库或跨语言依赖，优先考虑 Conda。

Linux 好用的小工具

fastfetch

帮助你快速查看系统的状态。

btcp

终端中的任务管理器。

exa

替代ls。

fd

命令行搜索工具，Rust编写，很快。

ripgrep

替代grep查找内容。

fzf

模糊搜索。

Linux 终端使用

通过代理访问 Clab

首先需要在物理机配置 ssh 设置：

进入 .ssh 文件夹：

```
1 ### macOS
2 cd .ssh
3
4 ### Windows
5 cd C:\\Users\\YourUserName\\.ssh
```

创建 `config` 文件并写入：

```
1 # 普通 SSH 登录配置
```



```

2  # 使用方式:
3  # ssh 10.129.245.149
4  Host 10.129.245.149
5      # 实际连接的远程服务器地址, 可以改成你的云主机 IP
6      HostName 10.129.245.149
7      # 默认登录用户名, 可以改成 fedora / ubuntu / root 等
8      User fedora
9
10 # 带远程端口转发的 SSH 登录配置
11 # 使用方式:
12 # ssh 10.129.245.149~proxy
13 Host 10.129.245.149~proxy
14     # 实际连接的远程服务器地址, 可以改成你的云主机 IP
15     HostName 10.129.245.149
16     # 默认登录用户名 User fedora
17     # 远程端口转发:
18     # 让远程服务器的 127.0.0.1:7897
19     # 转发到你本地电脑的 127.0.0.1:7897
20     #
21     # 常见用途:
22     # 如果你本地开了代理, 例如 Clash / HTTP proxy,
23     # 并监听在本地 7897 端口,
24     # 那么云主机可以通过 127.0.0.1:7897 使用你本地的代理。
25     #
26     # 格式:
27     # RemoteForward <远程端口> <本地地址>:<本地端口>
28     #
29     # 如果你的本地代理端口是 7890, 可以改成:
30     # RemoteForward 7890 127.0.0.1:7890
31     RemoteForward 7897 127.0.0.1:7897
32
33     # 如果端口转发失败, 就直接退出 SSH
34     # 避免“SSH 连上了, 但代理其实没转发成功”的情况
35     ExitOnForwardFailure yes
36     # 每 60 秒给服务器发一次保活包
37     # 防止 SSH 长时间空闲后被断开 ServerAliveInterval 60
38     # 如果连续 3 次保活没有响应, 就认为连接断了
39     # 配合上面的 60 秒, 大约 180 秒无响应后断开
40     ServerAliveCountMax 3

```

接下来需要在 Clab 机器中更改终端配置:

```

1  # 打开 bash 配置文件
2  nano ~/.bashrc
3
4  # Use the SSH-forwarded local HTTP proxy whenever it is reachable.
5  unset ALL_PROXY all_proxy HTTP_PROXY HTTPS_PROXY http_proxy https_proxy
6  if command -v timeout >/dev/null 2>&1 && command -v nc >/dev/null 2>&1 && timeout 1 nc -z
7  127.0.0.1 7897 >/dev/null 2>&1; then
8      export HTTP_PROXY="http://127.0.0.1:7897"
9      export HTTPS_PROXY="$HTTP_PROXY"
10     export http_proxy="$HTTP_PROXY"
11     export https_proxy="$HTTP_PROXY"
12 fi
13 ulimit -n 65535 2>/dev/null || true

```

Ctrl + O 写入配置文件保存, Ctrl + X 退出。

这样之后想要通过代理 ssh 连接 Clab 机器，首先在终端建立 proxy 通道：

```
1 | ssh -fN 10.129.245.149-proxy
```

之后正常连接：

```
1 | ssh 10.129.245.149
```

进入云主机后检查是否连接成功：

```
fedora ~
> echo $HTTP_PROXY
http://127.0.0.1:7897
v3.14.5
```

Linux 终端美化

[【双语+纯享】🔥2026终极Zsh配置指南：打造比华尔街还炫酷的终端体验!⚡](#)

Linux 默认终端为 bash (不同发行版可能不一样)

这里的“终端”严格来说分成两层：一层是你看到的窗口或远程 SSH 会话，另一层是里面真正负责解释命令的 shell。`bash` 和 `zsh` 都是 shell。

`bash` 是 Linux 上最常见的默认 shell，兼容性最好，服务器脚本和教程里大量命令都默认按 `bash` 编写。它的配置文件通常是：

```
1 | ~/.bashrc
```

如果只是想让 Clab 稳定可用，保留 `bash` 就足够。前面配置代理时修改的也是 `~/.bashrc`，这样每次登录 `bash` 都会自动加载这些设置。

`zsh` 是另一个常见 shell，macOS 现在默认使用 `zsh`。它的优点是补全、主题、插件生态更丰富，常和 Oh My Zsh、Starship 等工具一起使用，适合做终端美化。它的配置文件通常是：

```
1 | ~/.zshrc
```

简单来说：

- 想要稳定、少折腾：用 `bash`。
- 想要更好看的提示符、自动补全和插件：可以用 `zsh`。
- 改 `bash` 配置通常写 `~/.bashrc`。
- 改 `zsh` 配置通常写 `~/.zshrc`。

查看当前正在使用的 shell：

```
1 | echo $SHELL
```

查看当前命令解释器：

```
1 | echo $0
```

如果只是临时体验 `zsh`，可以直接运行：

```
1 | zsh
```

退出当前 shell:

```
1 | exit
```

不建议刚开始就随便修改默认 shell。对于 Clab 这种远程机器，优先保证能正常登录、能正常跑任务；终端美化可以后面再做。

在容器中运行 Linux

Docker

<https://docs.docker.com/get-started/>

WSL Container

<https://learn.microsoft.com/zh-cn/windows/wsl/wsl-container>

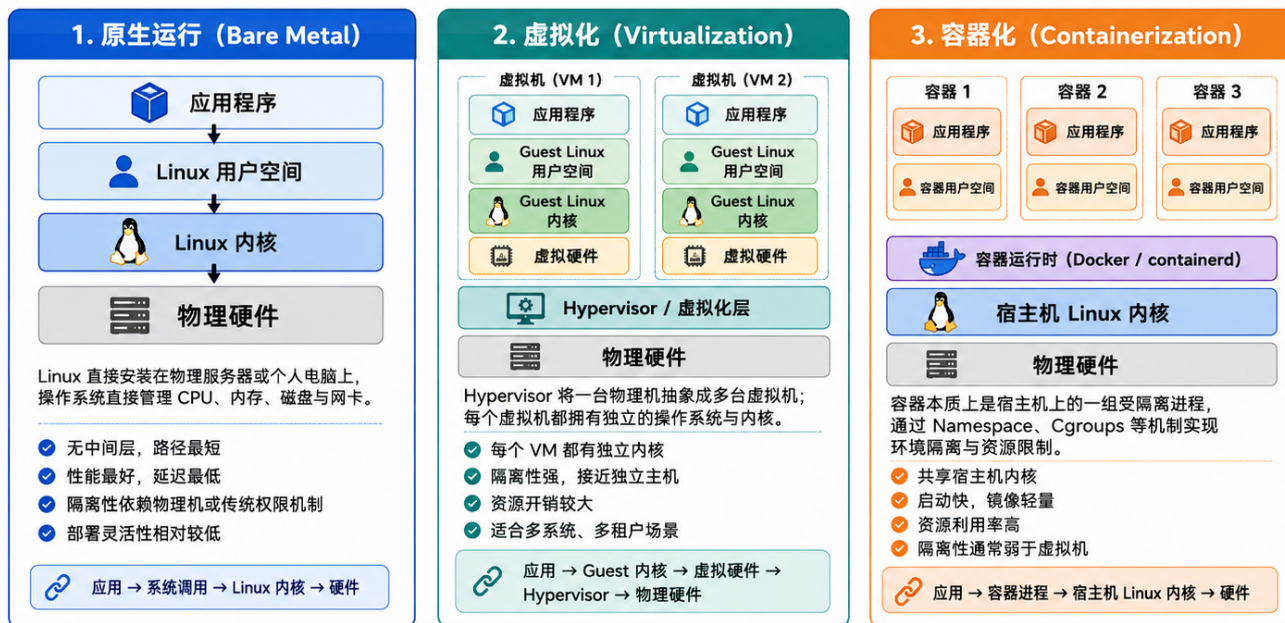
Apple Container

<https://github.com/apple/container>

本节仅作参考，单纯是因为我前几天看到WSL更新原生容器化方案才放上来的。

[WSL容器新功能详解](#) | [本地运行Linux容器全流程演示](#) | [Microsoft Developer](#)

Linux 运行方式对比：原生运行 vs 虚拟化 vs 容器化



核心差异总结

对比项	原生运行	虚拟化	容器化
内核使用	直接使用宿主 Linux 内核	每个 VM 各自拥有内核	共享宿主机内核
隔离粒度	最低	最强	中等
启动速度	正常	较慢	很快
资源开销	最低	最高	较低
性能损耗	最低	较高	较低
典型场景	单机部署、性能敏感	云主机、强隔离	微服务、DevOps、弹性扩容

容器可以理解成一套隔离的 Linux 环境。它和虚拟机不完全一样：虚拟机更像完整的一台电脑，容器更像在当前系统上启动一个独立的运行环境。对于刚开始折腾开发环境的同学，容器最大的好处是“不容易把本机环境弄乱”。比如你想试用某个命令行工具、安装一堆依赖、跑一个不确定会不会污染环境的脚本，就可以先放进容器里。

Claude Code 也可以放在容器中使用。这样做的意义不是让 Claude 变得更聪明，而是把它能看到、能修改的文件范围限制在容器里。尤其是在 Windows 上直接使用 agent 时，路径、权限、POSIX 命令和沙箱支持都容易出问题；用 Docker、WSL Container 或类似工具跑一个 Linux 容器，往往更接近真实服务器环境。

如果只是临时试一下，可以用 `docker run --rm` 创建一次性容器；但更实用的方式是创建一个持久化的 Claude Code 容器。这个容器不删除，Claude Code 只安装一次，登录状态和配置也保留在容器里。以后想用时，只需要重新进入这个容器。

先准备一个专门给容器使用的工作区。建议把希望 Claude Code 读取的项目都放在一个固定父目录下，例如：

```
1 mkdir -p "$HOME/claude-workspace"
2 mkdir -p "$HOME/datasets"
```

然后创建一个长期保留的容器：

```
1 docker run -dit \
2   --name claude-code-dev \
3   -v "$HOME/claude-workspace:/workspace" \
4   -v "$HOME/datasets:/data:ro" \
5   -w /workspace \
6   node:22-bookworm sleep infinity
```

这几行的含义是：

- `docker run`：创建并启动一个新容器。
- `-dit`：让容器在后台保持运行，同时支持之后进入交互终端。
- `--name claude-code-dev`：给容器起一个固定名字，之后可以用这个名字进入。
- `-v "$HOME/claude-workspace:/workspace"`：把本机的 `~/claude-workspace` 挂载到容器里的 `/workspace`。
- `-v "$HOME/datasets:/data:ro"`：把本机的 `~/datasets` 挂载到容器里的 `/data`，并设置为只读。
- `-w /workspace`：把容器的工作目录设为 `/workspace`。
- `node:22-bookworm`：使用带 Node.js 的 Debian Linux 镜像。
- `sleep infinity`：让容器保持运行，不会立刻退出。

第一次进入容器：

```
1 docker exec -it claude-code-dev bash
```

进入后安装 Claude Code：

```
1 npm install -g @anthropic-ai/claude-code
2 claude
```

这一步只需要做一次。只要你不删除这个容器，容器内部安装的 Claude Code、登录状态、shell 配置和临时文件都会保留。

以后想再次使用，不需要重新 `docker run`，只需要：

```
1 docker start claude-code-dev
2 docker exec -it claude-code-dev bash
3 claude
```

如果暂时不用，可以停止容器：

```
1 docker stop claude-code-dev
```

停止不会删除容器，下次 `docker start claude-code-dev` 后还能继续用。真正删除容器的是：

```
1 docker rm claude-code-dev
```

不要随便执行删除命令。容器删掉后，容器内部安装的 Claude Code 和登录状态也会一起消失。

挂载目录是这里最关键的 concept。容器默认看不到你的本机文件夹；只有通过 `-v` 挂载进去的目录，Claude Code 才能读取或修改。上面的例子里，容器内的 `/workspace` 对应本机的 `~/claude-workspace`。如果你把项目放到：

```
1 ~/claude-workspace/my-project
```

那么进入容器后就可以：

```
1 cd /workspace/my-project
2 claude
```

如果你在 Claude Code 中让它“读取当前项目”“修改这个脚本”，它实际操作的是容器里的 `/workspace/my-project`，而这个目录又对应本机的 `~/claude-workspace/my-project`。因此，容器中的修改会同步反映到本机文件夹。

需要注意：Docker 不能给一个已经创建好的普通容器随时追加新的 `-v` 挂载。也就是说，如果你创建容器时只挂载了 `~/claude-workspace`，后来想让它访问另一个完全不在这个目录下的文件夹，通常要重新创建容器，或者一开始就挂载一个更合适的父目录。比较推荐的做法是固定一个 `~/claude-workspace`，把需要给 Claude Code 处理的项目都放进去。

如果你确实经常要挂载不同目录，可以把 Claude Code 做成一个自定义镜像，再用 volume 保存 Claude 的配置。这样每次为不同目录创建新容器时，不需要重新下载 Claude Code，登录信息也可以复用。

先新建一个 `Dockerfile`：

```
1 FROM node:22-bookworm
2 RUN npm install -g @anthropic-ai/claude-code
3 WORKDIR /workspace
4 CMD ["sleep", "infinity"]
```

构建镜像：

```
1 docker build -t claude-code-dev:latest .
```

以后在任意项目目录下，可以创建一个挂载当前目录的容器：

```
1 docker run -dit \
2   --name claude-code-current \
3   -v "$PWD:/workspace" \
4   -v claude-code-home:/root/.claude \
5   -w /workspace \
6   claude-code-dev:latest
```

其中 `claude-code-home:/root/.claude` 是 Docker volume，用来保存 Claude Code 的登录信息和配置。换项目时可以删除旧容器、重新创建新容器，但镜像和 volume 还在，所以不用重新安装 Claude Code，也不一定需要重新登录。

如果只想让 Claude Code 读取资料，而不希望它修改，可以把目录以只读方式挂载。上面的 `/data` 就是这样：

```
1 -v "$HOME/datasets:/data:ro"
```

其中 `:ro` 表示 read-only。这样 Claude Code 可以读实验数据、总结文件结构、解释结果，但不能直接修改 `/data` 里的文件。对于科研任务，这是比较稳妥的习惯：代码目录可以允许修改，原始数据目录尽量只读。

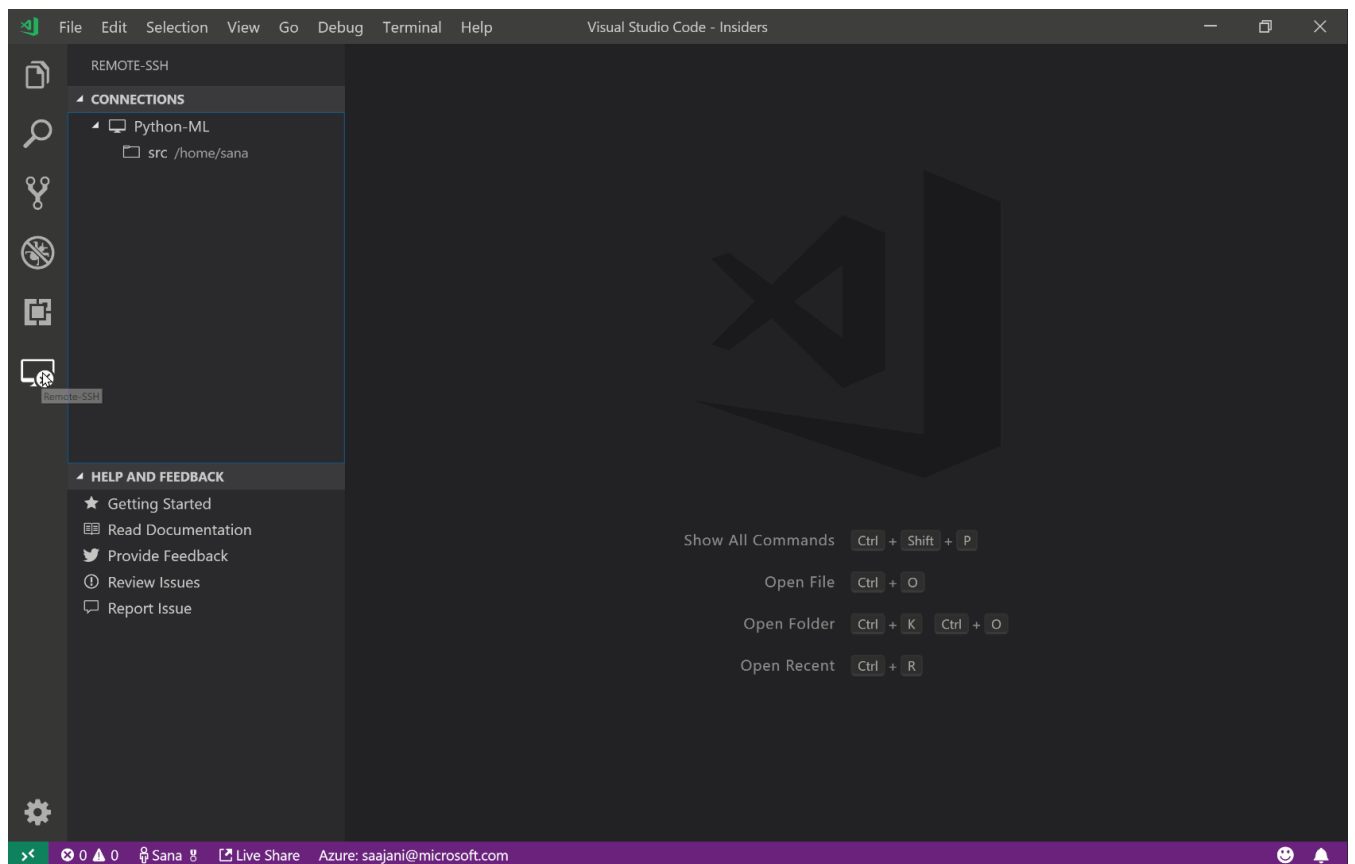
需要注意，“本机目录”指的是运行 Docker daemon 的那台机器上的目录。如果你在自己的笔记本上运行 Docker，`$PWD` 是笔记本上的目录；如果你 SSH 到 Clab 云主机后在云主机里运行 Docker，`$PWD` 就是 Clab 云主机上的目录，而不是你笔记本上的目录。Claude Code 只能看到容器内文件系统，以及你用 `-v` 明确挂载进去的目录。

不要一上来就把整个 home 目录或系统根目录挂进去，例如不要随便使用 `-v "$HOME:/host-home"` 或 `-v "":"/host"`。这样虽然方便，但等于把大量私人文件和系统文件暴露给 agent，误操作风险很高。更推荐的做法是：固定一个专门的工作区目录，只把本次任务真正需要的项目放进去。

VS Code 和 Clab 协同

通过 Remote-SSH 插件连接 Clab

在插件市场搜索 `Remote-SSH` 并安装。



首次连接需要较长时间下载vscode-server。

在 VS Code Server 上使用代理

可以在 `~/.vscode-server/data/Machine/settings.json` 中添加 proxy:

```
1  "http.proxy": "http://127.0.0.1:7897",
2  "http.proxySupport": "on",
3  "http.systemCertificates": true,
4  "http.proxyStrictSSL": false,
```

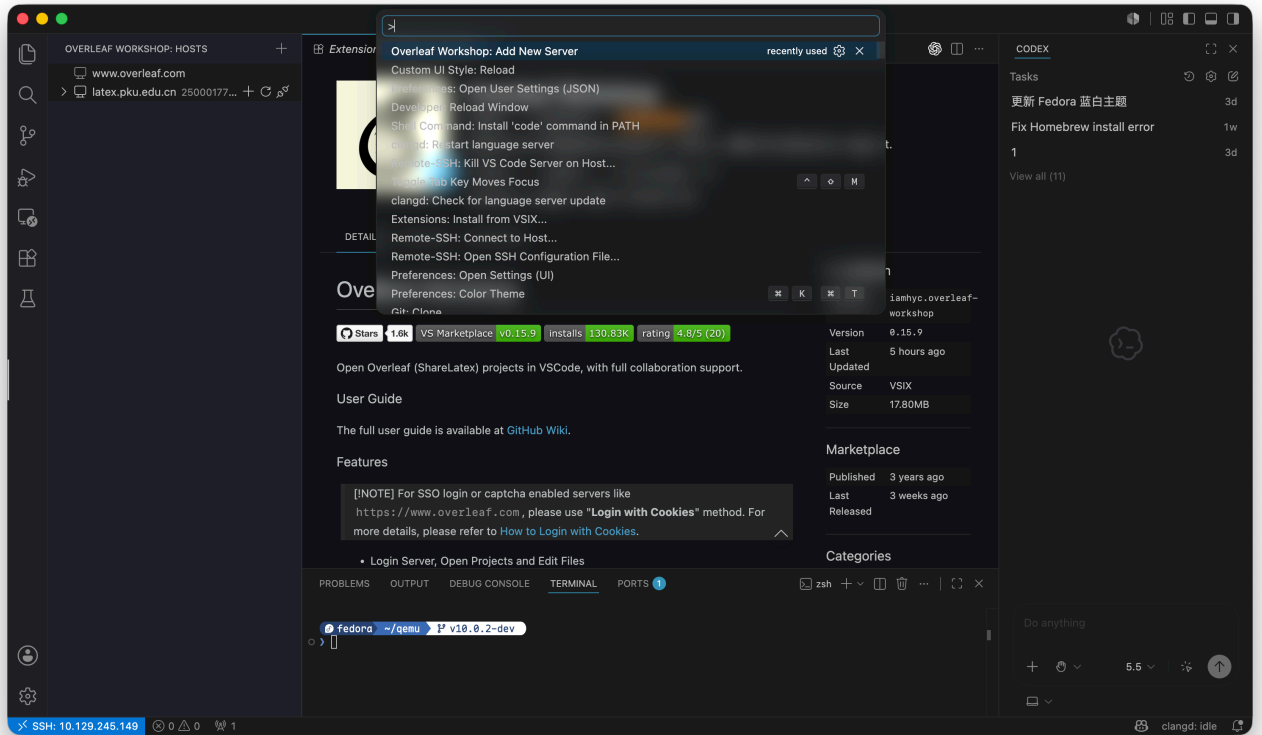
PKUTeX

[PKUTeX](#) 是一个线上 LaTeX 编辑器，支持实时编辑和编译，最大的优点是不需要本地化的编译环境，极大节省了本地存储空间，同时编译效率很高，推荐大家使用。

可以通过插件连接 VS Code 进行编辑和版本管理，同时可以连接 Agent 实现 Vibe TeXing。

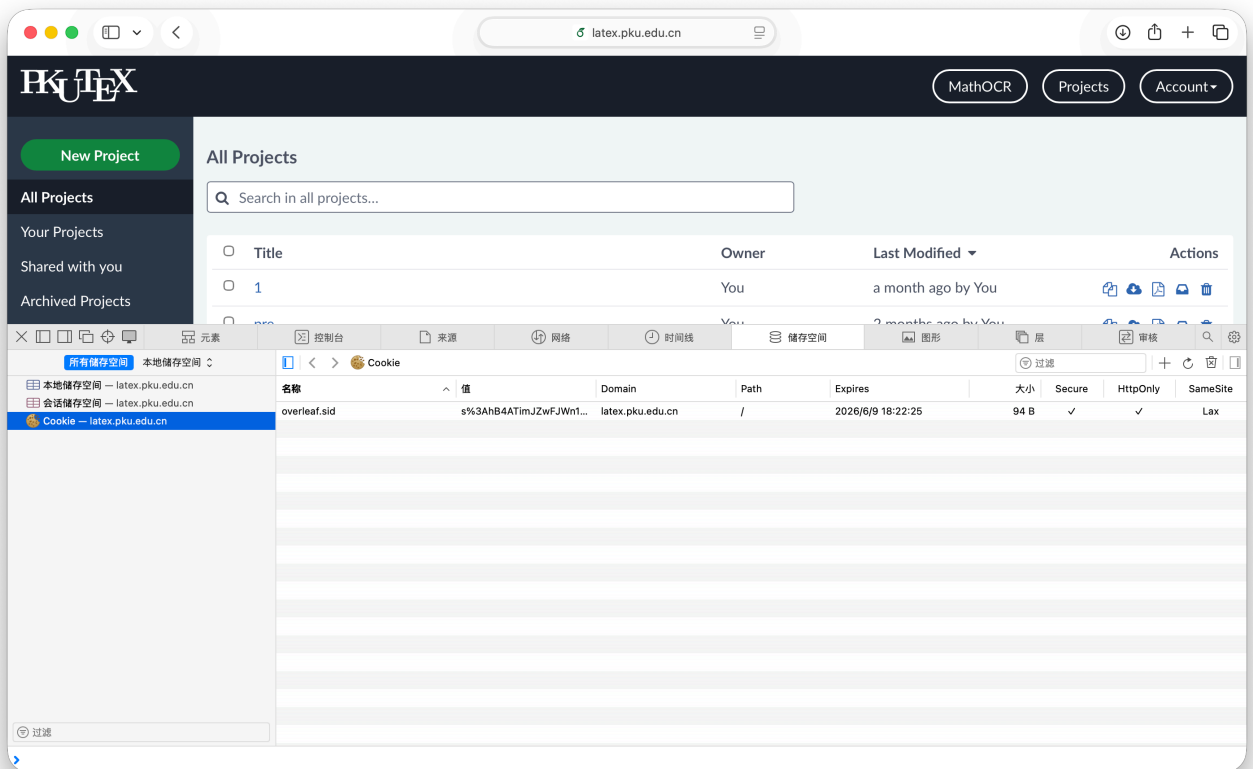
首先在插件市场下载 `Overleaf Workshop`。

然后添加 PKUTeX 的服务器：`latex.pku.edu.cn`



需要 Cookie 登录：

Safari: 右键-检查元素 -> 储存空间 -> Cookie 复制 value，填写时带上 `overleaf.sid=...`；

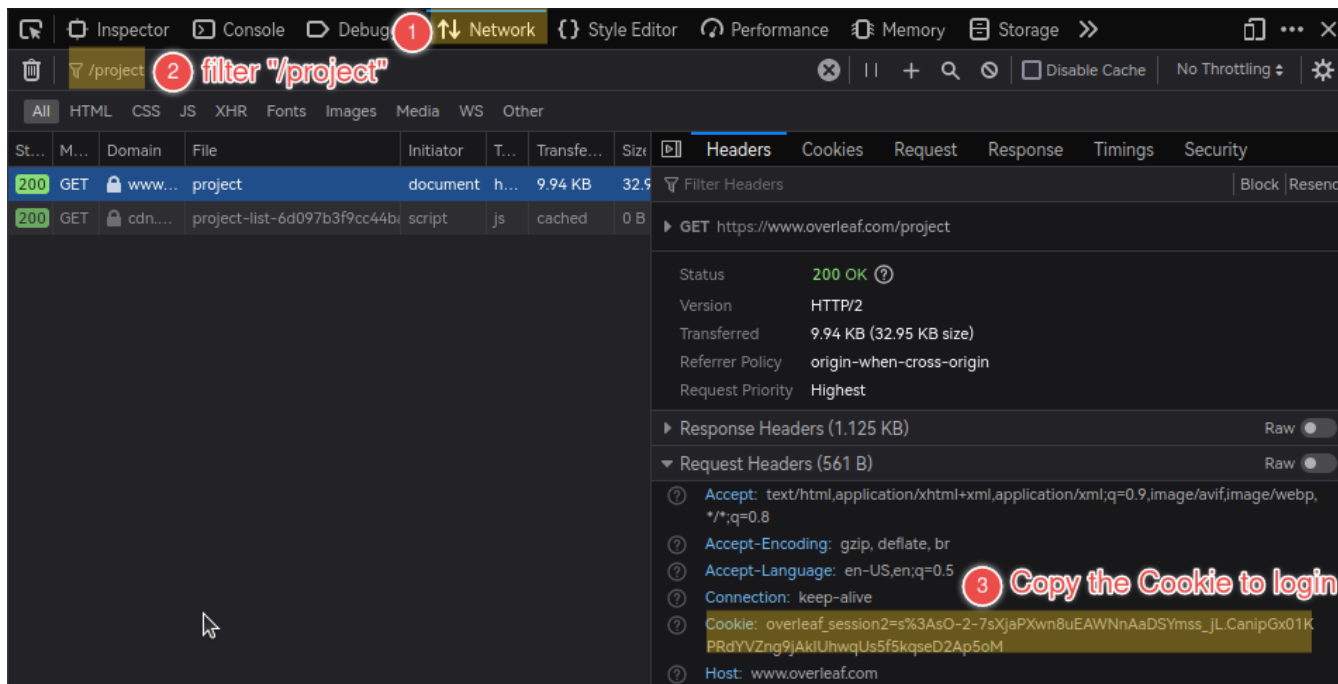


Edge:

In an already logged-in browser:

1. Open "Developer Tools" (usually by pressing `F12`) and switch to the "Network" tab;
Then, navigate to the Overleaf main page (e.g., `https://latex.pku.edu.cn`) in the address bar.
2. Filter the listed items with `/project` and select the exact match.
3. Check the "Cookie" under "Request Headers" of the selected item and copy its value to login.

The format of the Cookie value would be like: `overleaf.sid=...`



AI Agent 简介：以 Claude Code 为例

强烈建议在 Linux 或 macOS (darwin) 环境下使用 agent，Windows 对 posix 的支持近乎没有，使得 agent 更新往往落后一到两个大版本，同时沙箱环境适配很差，环境隔离几乎没有，很容易损坏本地文件系统。

How Claude Code Works

Documentation Index: <https://code.claude.com/docs/en/how-claude-code-works>

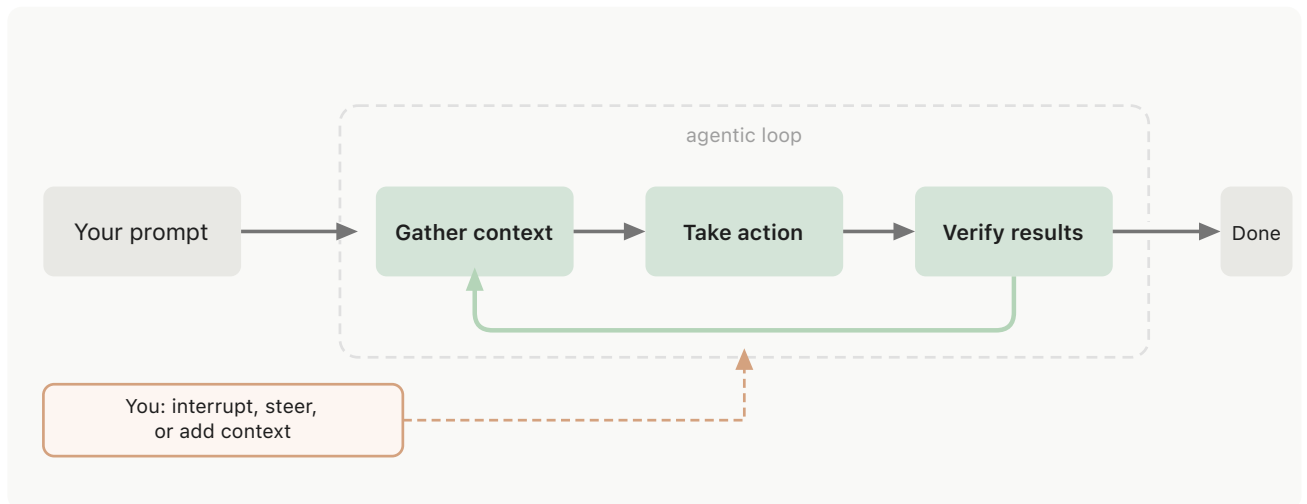
使用 Claude Code 文档时，可以先读取 `overview` 作为索引，再根据索引继续打开具体页面。这样能避免只凭记忆查文档，也方便发现 `skills`、`MCP`、`hooks`、`subagents`、`sessions` 等相关页面。

Claude Code 可以理解成运行在终端里的 agent framework。模型负责理解任务、拆解步骤和推理下一步；Claude Code 负责提供工具、上下文管理和执行环境，让模型能够真正读取文件、运行命令、编辑代码并验证结果。

Agent Loop

Claude Code 的核心工作方式是一个循环：

1. **收集上下文**：读取文件、搜索代码、查看 git 状态、理解项目结构和用户要求。
2. **采取行动**：编辑文件、创建文档、运行 shell 命令、调用 MCP 或其他外部工具。
3. **验证结果**：运行测试、检查输出、读取错误信息，再决定是否继续修正。



这三个阶段不是严格分开的。比如修复一个测试失败时，Claude 可能会先运行测试，读取报错，搜索相关代码，修改实现，再运行测试验证。每一步工具调用都会把新信息反馈给模型，影响下一步决策。用户也可以随时中断、补充上下文或要求换方向。

Models and Tools

Claude Code 中的模型负责推理。Sonnet 适合大多数编码任务，Opus 更适合复杂架构判断；可以在会话中用 `/model` 切换，或启动时指定模型。

工具让 Claude Code 从“只会回答”变成“能做事”。常见工具能力包括：

- 文件操作：读取、编辑、创建、移动文件。
- 搜索：按文件名、模式或正则搜索项目内容。
- 执行：运行测试、构建、git、包管理器和系统命令。
- 网络：查文档、搜索网页、读取外部资料。
- 代码智能：查看类型错误、跳转定义、查找引用等。

在这些基础工具之上，还可以继续扩展：用 skills 增加特定 workflow 知识，用 MCP 连接外部服务，用 hooks 自动化流程，用 subagents 把长任务拆给独立上下文处理。

Claude 可以访问什么

在某个目录运行 Claude Code 时，它通常可以访问：

- 当前项目目录及其子目录中的文件。
- 当前终端能运行的命令。
- git 状态、分支、未提交更改和历史提交。
- `CLAUDE.md` 中写入的项目规则和长期说明。
- 自动记忆中的项目模式和用户偏好。
- 已配置的 MCP servers、skills、subagents、Claude in Chrome 等扩展。

如果 Claude Code 运行在容器里，它的可见范围还会受到容器边界限制：没有被挂载进容器的本机文件夹，它通常看不到；被挂载为只读的目录，它可以读取但不能直接修改。

这也是 Claude Code 和只看当前文件的代码补全工具不同的地方：它能跨文件理解项目，基于终端和工具完成端到端任务。

环境、会话和上下文

Claude Code 可以在本地、云端或远程控制环境中运行；也可以通过终端、桌面应用、IDE 扩展、网页、Slack 或 CI/CD 管道使用。界面不同，但底层 agent loop 相同。

会话会保存在本地，新的会话默认没有旧对话历史。要持久保存项目约定，应写入 `CLAUDE.md`，不要只依赖聊天上下文。长会话中上下文窗口会逐渐填满，Claude 会自动压缩旧工具输出和对话摘要；如果有必须长期保留的规则，应放进持久文件或 memory。

管理上下文时，skills 和 subagents 很有用：skills 按需加载完整说明，subagents 拥有独立上下文，完成后只把摘要返回主会话，适合处理长任务或隔离探索过程。

安全机制

Claude Code 主要依赖两类安全机制：

- **检查点**：文件被编辑前会保存快照，出错时可以撤销。
- **权限模式**：控制 Claude 是否能自动编辑文件或运行命令。

常见权限模式包括默认模式、自动接受编辑、Plan Mode 和自动模式。对于有外部副作用的操作，比如数据库、部署、提交代码或删除文件，应优先让 Agent 说明计划，再批准执行。

使用建议

更有效的使用方式是吧 Claude Code 当成一个可以委派的工程同事：给清楚目标、约束、相关目录和验证方式，但不必逐条指定它读哪些文件。复杂任务可以先用 Plan Mode 让它探索和制定计划，再确认实现。实现类任务最好提供可验证标准，例如测试用例、预期输出或截图。过程中如果方向不对，可以直接中断并补充说明，而不是重新开始。

Quick Start

[Claude Code Quick Start](#)

让 Claude Code 调用 DeepSeek API

API Documentaion:

https://api-docs.deepseek.com/zh-cn/quick_start/agent_integrations/claude_code

配置文件在 `~/.claude/settings.json`，添加配置项：

```
1  "env": {
2    "ANTHROPIC_BASE_URL": "https://api.deepseek.com/anthropic",
3    "ANTHROPIC_AUTH_TOKEN": "<your-api-key>",
4    "ANTHROPIC_MODEL": "deepseek-v4-pro[1m]",
5    "ANTHROPIC_DEFAULT_OPUS_MODEL": "deepseek-v4-pro[1m]",
6    "ANTHROPIC_DEFAULT_SONNET_MODEL": "deepseek-v4-pro[1m]",
7    "ANTHROPIC_DEFAULT_HAIKU_MODEL": "deepseek-v4-flash",
8    "CLAUDE_CODE_SUBAGENT_MODEL": "deepseek-v4-flash",
9    "CLAUDE_CODE_EFFORT_LEVEL": "max"
10 }
```

或者用 [CC Switch](#).

MCP：给 Agent 接外部工具

MCP 是 Model Context Protocol 的缩写，可以理解为 Agent 的“外设接口”。没有 MCP 时，Agent 主要依赖自己能直接看到的文件和你粘贴给它的内容；接入 MCP 后，Agent 可以通过统一接口读取或操作外部系统，例如 Notion、Obsidian、GitHub、数据库、浏览器或课程文档库。

MCP 的作用不是让模型“自动变聪明”，而是让它拿到更准确的上下文，并能使用更合适的工具。比如让 Agent 总结 CLab v2，如果没有 MCP，它只能依赖你手动复制的内容；如果接入了文档库 MCP，它就可以直接读取相关文档，再基于真实材料进行整理。

一个 MCP server 通常会暴露若干工具，例如搜索页面、读取文档、创建页面、更新笔记。使用 MCP 时要注意权限：读文档、查资料通常风险较低；创建文档、修改数据库、提交代码这类写操作风险更高。比较好的使用方式是：让 Agent 先说明它要读取或修改什么，再允许真正的写操作。

MCP 接入 Protocol: Codex + Obsidian/Outline

目标：让 Agent 通过 MCP 读取或操作外部知识库，并把认证信息持久化。配置完成后，Agent 不再依赖手动复制材料，而是可以通过工具直接搜索、读取、创建或更新文档。

前置条件：Obsidian/Outline 服务本身要能访问；`~/.codex/config.toml` 中的 MCP server 要启用；修改配置后需要重启 Codex，让当前线程重新加载工具。

1. 配置 Obsidian

Obsidian 使用 Local REST API with MCP 插件。它默认提供 HTTPS 端口 `27124`，但本地证书是自签名证书；如果没有把证书加入系统信任，Codex 在 MCP `initialize` 阶段可能握手失败。为了简化本机使用，可以启用插件的 insecure HTTP server，只监听 `127.0.0.1:27123`：

```
1 [mcp_servers.obsidian]
2 enabled = true
3 url = "http://127.0.0.1:27123/mcp/"
4
5 [mcp_servers.obsidian.http_headers]
6 Authorization = "Bearer <Obsidian API key>"
```

验证端点：

```
1 curl -i http://127.0.0.1:27123/mcp/
```

不带 token 时返回 `401 Authorization required` 是正常的，说明服务可达但请求未授权。真正接通的标准是：Codex 中出现 `mcp_obsidian`，并且 `vault_list`、`vault_read`、`tag_list` 这类只读工具能成功调用。

如果坚持使用 HTTPS，则需要把 Obsidian 插件提供的 certificate 加入 macOS Keychain，并设为 Always Trust；否则 Codex 可能因为不信任证书而无法加载工具。

2. 配置 Outline

Outline 是远端 Streamable HTTP MCP，认证走 OAuth。为了避免每次调用都手动登录，把 MCP OAuth 凭据保存到系统 Keychain：

```
1 mcp_oauth_credentials_store = "keyring"
2
3 [mcp_servers.outline-lcpu]
4 url = "https://outline.lcpu.dev/mcp"
5 scopes = ["read", "write"]
6 oauth_resource = "https://outline.lcpu.dev/mcp"
```

注意：`mcp_oauth_credentials_store = "keyring"` 必须放在 `config.toml` 顶层，不能写进 `[mcp_servers.outline-lcpu]` 里面。

配置后登录一次：

```
1 codex mcp login outline-lcpu --scopes read,write
```

完成浏览器授权并重启 Codex 后，正常情况下 Codex 会复用 refresh token。真正接通的标准是：Codex 中出现 `mcp_outline_lcpu`，并且 `list_collections`、`list_documents` 这类只读工具能成功调用。

3. 排障顺序

先看配置是否被识别：

```
1 | codex mcp list
2 | codex mcp get obsidian
3 | codex mcp get outline-lcpu
```

再看服务是否可达：本地服务用 `curl` 检查端口，远端 OAuth 服务检查 `.well-known` metadata。最后看当前线程是否暴露对应的 `mcp_...` 工具，并优先用只读工具验证。

需要区分两个状态：`enabled` 只说明配置启用；`mcp_...` 工具出现并能调用，才说明 MCP server 已经真正加载成功。